

PulseCraft AI – Gesprächs-Coach — Vollständige technische Beschreibung

Stand: 2026-06-13 · Branch `main` (nach Merge von Quality-Loop Q2) · Live: <https://pulsecraft-coach.azurewebsites.net>

Diese Fassung beschreibt die App umfassend — von der Produkt-Idee über die technische Architektur, die KI-Analysekette und das Cosmos-Datenmodell bis zu den Sicherheits-, Kosten- und Qualitäts-Schichten. **Alle Architektur-Aussagen sind aus dem laufenden Code extrahiert** und mit `Datei:Zeile` belegt. Zielgruppe ist ein Senior-Entwickler, der die App prüfen und bewerten soll. Die Abschnitte 20 (Schwächen/Risiken) und 21 (Prüf-Leitfaden) sind bewusst schonungslos: sie konsolidieren rund 100 konkrete Review-Findings aus einem vollständigen Code-Audit. Wo der Code etwas *nicht* leistet, steht das explizit.

Inhaltsverzeichnis

1. Überblick & Produkt-Zweck
2. Reifegrad & ehrlicher Ist-Stand
3. Tech-Stack & Dependencies
4. Gesamtarchitektur — der Analyse-Lebenszyklus (Schritt für Schritt)
5. Datenmodell (Cosmos-Container + Kern-Interfaces)
6. Persistenz-Muster & Cosmos-Eigenheiten
7. Authentifizierung & Session
8. API-Surface (~15 Routen) + Fehler-/Statusmodell
9. Die KI-Schicht — Flows & Prompt-Kette
10. Schema-Forced Reasoning & Prompt-Design
11. RAG / Pinecone-Retrieval
12. Kompetenzmodell C1–C10
13. Kosten- & Missbrauchsschutz (Cost-Cap, Rate-Limit, Prompt-Guard)
14. Qualitäts-Layer (deterministische Checks + Quality-Loop + LLM-Judge)
15. Frontend (App Router, Client-Komponenten, Transcript-Tooling)
16. Internationalisierung (7 Sprachen)
17. LinkedIn-Integration
18. Sicherheitsmodell
19. Infrastruktur & Deployment (Azure)
20. Bekannte Schwächen, Risiken & Roadmap
21. Prüf-Leitfaden für den Reviewer
22. Anhang A — ENV-Variablen-Referenz
23. Anhang B — Zentrale Prompts & Schemas (verbatim)

1. Überblick & Produkt-Zweck

Der **PulseCraft AI Gesprächs-Coach** ist eine Next.js-Webanwendung (App Router), die **Transkripte aus Mitarbeitendengesprächen** (Führungskraft ↔ Mitarbeiter:in) analysiert und der Führungskraft strukturiertes, KI-gestütztes Coaching-Feedback liefert. Optimiert wird ein **generischer Analysator** für Führungsgespräche, nicht eine einzelne Auswertung.

Der rote Faden eines Laufs („Run“):

```
Transkript (PDF-Upload oder Texteingabe)
→ Client-seitige Bereinigung (Teams-Format) + Anonymisierung (Privacy-Mode default-on)
→ POST /api/analyze
  └─ generateDynamicFeedback (RAG-Retrieval aus Pinecone → generateTailoredFeedback)
    └─ → summary, strengths[], improvements[], rewrites[], riskFlags[], scores
      └─ scoreCompetencies (10 Leadership-Kompetenzen C1-C10, Schema-Forced Reasoning)
→ deterministische Qualitäts-Checks (Evidenz-Grounding, Score⇒Evidenz, Rewrite≠Original, Fl
→ POST /api/runs/save (Azure Cosmos DB)
→ Redirect /runs/{sessionId}/{runId} → ReportDashboard
  (ScoreRing, Stärken/Verbesserungen/Risiken, Rewrites, Kompetenz-Panel,
  5-Sterne-Rating, JSON-Download, 7-Tage-ICS-Erinnerung, optional LinkedIn-Post)
```

Didaktischer Kern: zehn Leadership-Kompetenzen (C1–C10), bewertet auf einer Skala **1–4** (oder `null` = „nicht beobachtbar“). Jede Bewertung muss durch **wörtliche, anonymisierte Zitate** aus dem Transkript belegt sein — das Modell wird gezwungen, erst Evidenz zu sammeln und dann zu urteilen (`src/ai/flows/score-competencies.ts:17–48`).

Drei Leitprinzipien, die jede Designentscheidung prägen:

- **DSGVO / Akzeptanz in Deutschland.** Auth, Daten, Hosting und Secrets liegen in der EU (Azure West Europe, Microsoft-AVV). Transkripte werden zusätzlich client-seitig anonymisiert.
- **Qualitätserhalt beim Modellwechsel.** Die Migration von Firebase auf Azure behält bewusst **Gemini** als LLM und **Pinecone** (EU-Index) als RAG — beides als Folgeschritt „M7“ offen, damit die Analysequalität nicht durch einen erzwungenen Modellwechsel sinkt.
- **Messbare statt blinde Qualität.** Ein eigener Quality-Loop (deterministische Checks + neutraler LLM-Judge + Golden-Set-Regression) misst den Generator reproduzierbar (Abschnitt 14).

2. Reifegrad & ehrlicher Ist-Stand

Live und verifiziert (Azure App Service `pulsecraft-coach`): Microsoft-Login (Entra ID), Transkript-Analyse end-to-end, Run-Persistenz + Historie, Report mit Rating/Download, Kompetenz-Scoring, deterministische Qualitäts-Checks, Cost-Cap pro User/Tag. Negativ-verifiziert: geschützte API-Routen liefern ohne Session **401**; Run-Detailseiten leaken ohne Auth keine Daten (IDOR-Fix); alte Firebase-Hosting-Endpunkte → 404.

Branch-/Deploy-Realität — wichtig für die Bewertung:

- `main` enthält den vollständigen migrierten Stand inkl. der Quality-Hardening-Wellen Q1 + Q2.
- Git-Remote: `github.com/thiemann-j-hub/AI_Coach`.
- **Deployment ist manuell** (Standalone-ZIP via `az webapp deploy`) — **es gibt keine CI/CD-Pipeline**.
- Der Quality-Harness (`scripts/quality/`) ist ein **lokales** Tool, nicht in einen Pre-Push-Gate eingebunden (`docs/quality/README.md`).

Bewusste, dokumentierte Lücken (Details Abschnitt 20): in-memory Rate-Limit (Multi-Instance-schwach), Gemini/Pinecone außerhalb EU (M7 offen), README noch auf Firebase, zwei divergente Kompetenz-Modelle im Code, LinkedIn-Credentials noch Platzhalter, kein Token-Refresh für LinkedIn.

Code-Umfang & -Disziplin: 105 TypeScript-Dateien unter `src/` ; durchgängig `strict` -Mode; Zod-Validierung auf allen Routen; `next.config.ts` ohne `ignoreBuildErrors` (der Build ist ein echtes Qualitäts-Gate); 10 Module mit `server-only` -Guard.

Migrations-Historie: Die App durchlief eine vollständige Firebase→Azure-Migration in den Phasen M0–M6 (Code-Vorbereitung, Azure-Provisionierung, Auth-Migration, Daten-Migration, Build/Deploy, Live-Verifikation, Firebase-Rückbau), dokumentiert in `docs/azure-migration/PLAN.md` . Danach zwei Quality-Hardening-Wellen: **Q1** (Cost-Cap, Transkript-Grounding, Schema-Forced Reasoning, deterministische Validatoren) und **Q2** (Golden-Set + Regression-Harness + neutraler LLM-Judge).

3. Tech-Stack & Dependencies

Schicht	Technologie	Beleg
Framework	Next.js 15.5.9 (App Router) + React 19.2.1	<code>package.json:45-50</code>
Sprache	TypeScript 5.x, <code>strict: true</code> , <code>moduleResolution: bundler</code> , Pfad <code>@/*</code> → <code>./src/*</code>	<code>tsconfig.json:1-27</code>
Auth	NextAuth v5 (<code>next-auth@5.0.0-beta.31</code>) + Microsoft Entra ID (OIDC)	<code>src/auth.ts:1-36</code>
DB	Azure Cosmos DB for NoSQL (<code>@azure/cosmos@4</code>) — Container <code>users/sessions/runs/usage</code>	<code>src/lib/cosmos.ts:1-96</code>
LLM	Google Gemini via Genkit (<code>genkit@1.20</code> , <code>@genkit-ai/google-genai</code> , <code>@genkit-ai/next</code>), Modell <code>googleai/gemini-2.5-flash</code>	<code>src/ai/genkit.ts:1-7</code>
Vektorsuche	Pinecone (REST „Search with text“, EU-Index <code>gcp-europe-west4</code>)	<code>src/lib/pinecone.ts:1-40</code>
Bild-Gen (optional)	NanoBanana-Wrapper (<code>src/lib/nanobanana.ts</code>), <code>NANOBANANA_MODEL</code>	<code>.env.example:40</code>
Styling	Tailwind CSS 3.4 + 25 Radix-UI-Komponenten + CVA + <code>tailwind-merge</code> + <code>tailwindcss-animate</code>	<code>package.json:21-65</code>
Forms/Validierung	<code>react-hook-form 7.54</code> , <code>@hookform/resolvers 4.1</code> , Zod 3.24	<code>package.json:20,52,55</code>
PDF	<code>pdfjs-dist@5.4</code> — rein clientseitiges Parsing, Worker aus <code>/public/pdf.worker.min.js</code>	<code>src/lib/pdf/parsePdfToText.ts</code>
i18n	Eigene Lösung, 7 Locales (DE Source-of-Truth), Cookie <code>NEXT_LOCALE</code>	<code>src/i18n/config.ts:1-24</code>
Server-Guard	<code>server-only@0.0.1</code> (devDep, Compile-Time- Check)	<code>package.json:64</code>
Runtime	Node 22.x	<code>package.json:69</code>
Hosting	Azure App Service (Linux, B2, West Europe), <code>output:</code> <code>'standalone'</code> , ZIP-Deploy	<code>next.config.ts</code> , <code>docs/azure-migration/PLAN.md</code>
Secrets	Azure Key Vault <code>coach-kv-20f84</code> + System- Managed-Identity	<code>docs/azure-migration/PLAN.md:84</code>
Tooling	<code>tsx</code> (One-off-Skripte), <code>dotenv</code> (devDep)	<code>package.json:60</code>

npm-Scripts (`package.json:6-13`):

Script	Befehl	Zweck
dev	next dev --turbo pack -p 9002	lokaler Dev-Server
build	next build	Production-Build (standalone)
start	next start	Production-Server
typecheck	tsc --noEmit	Typprüfung
quality:regression	node --conditions=react-server --import tsx scripts/quality/run-regression.ts	Qualitäts-Regression gegen Golden-Set
genkit:dev / genkit:watch	genkit start -- tsx src/ai/dev.ts	Genkit-Flow-Debugging

Die `--conditions=react-server`-Auflösung im Quality-Script mappt das `server-only`-Paket auf seine No-op-Variante, damit der Flow außerhalb des Next-Bundlers (in plain Node/tsx) lauffähig ist.

4. Gesamtarchitektur — der Analyse-Lebenszyklus (Schritt für Schritt)

Die App ist eine reine Next-App-Router-Anwendung: **Server Components + Route Handler** für alles Sensible (Auth, DB, LLM, Secrets), **Client Components** für die interaktiven Flächen. Es gibt keinen separaten Backend-Dienst — die `src/app/api/**`-Route-Handler *sind* das Backend; State-Server ist Cosmos DB.

Sequenz eines Laufs (mit konkreten Aufrufen):

- Eingabe** (`src/app/analyze/AnalyzeClient.tsx`): Der Nutzer lädt ein PDF hoch oder fügt Text ein. PDF-Parsing passiert **vollständig im Browser** (`parsePdfToText` , `src/lib/pdf/parsePdfToText.ts:56-107`) — kein Server-Upload, der Worker liegt lokal in `/public/pdf.worker.min.js` , Limits `maxPages=30` , `maxChars=250000` . Optional folgt ein Teams-Format-Cleanup (`cleanTeamsTranscript`) und automatische Sprecher-Erkennung (`detectSpeakers` , `src/lib/transcript-utils.ts:38-81`).
- Anonymisierung** (Privacy-Mode, default-on): Vor dem Senden ersetzt `sanitizeTranscript` (`src/lib/transcript-utils.ts:94-140`) E-Mails → `[EMAIL]` , URLs → `[URL]` , Telefonnummern → `[TEL]` , gequotete Projekt-/Kundennamen → `[PROJEKT]` / `[KUNDE]` , Sprecher → „Führungskraft“/„Mitarbeiter:in“/„Person N“, sowie nutzerdefinierte Begriffe → `ANON_n` . Die Wortgrenzen verwenden Unicode-aware Lookbehind (`(?<![\p{L}\p{N}_])`) mit Split-Fallback für ältere Engines.
- Analyse** (`POST /api/analyze` , `src/app/api/analyze/route.ts:67-216`): `requireAuth` (401-Gate) → **IP-Rate-Limit** (10/min) → **Cost-Cap** (Cosmos, pro User) → Zod-Validierung → **parallel** (`Promise.allSettled`) `generateDynamicFeedback` (Feedback+RAG) und `scoreCompetencies` (C1–C10) → Label-Substitution + `normalizeScore` → **deterministische Qualitäts-Checks** → Response `{ ok, result }` .
- Persistenz** (`POST /api/runs/save` , `src/app/api/runs/save/route.ts:117-234`): `requireAuth` → 20/min → Zod → **checkSessionOwnership** **VOR dem Write** → `upsertSession` + `createRun` (UUID). Das Transkript wird nur gespeichert, wenn `storeTranscript === true` .
- Anzeige** (`/runs/{sessionId}/{runId}`): `page.tsx` ist nur ein `AuthGuard`-Wrapper; `RunDetailClient.tsx:107-133` lädt den Run **clientseitig** über `GET /api/runs/get` , das `requireAuth`

und `checkSessionOwnership` erzwingt (der IDOR-Fix, Abschnitt 18). Der `ReportDashboard` rendert das Ergebnis.

Degradations-Prinzip: Im `/api/analyze` ist das Basis-Feedback **Pflicht** (Flow-Reject → Request bricht mit 500 ab), das Kompetenz-Scoring **optional** (Reject → sichtbares `competency_error` + Fallback auf 10 Default-Kompetenzen mit `score=null`). Die Qualitäts-Checks sind **nie blockierend** (Fehler werden geloggt, `quality_notes` bleibt ggf. leer). Diese Asymmetrie (`route.ts:105-208`) ist bewusst: Feedback ist das Produkt, Scoring + Checks sind Anreicherung.

5. Datenmodell (Cosmos-Container + Kern-Interfaces)

Cosmos DB `coach` ist das System-of-Record. Zugriff **ausschließlich serverseitig** über `src/lib/cosmos.ts` (Endpoint + Key aus Key Vault); es gibt **keine Client-SDKs** und **keine Firestore-Rules** mehr — der gesamte Zugriffsschutz liegt in den Route-Handlern + Ownership-Checks.

Container (`src/lib/cosmos.ts:33-46` , `src/lib/server/cost-cap.ts:42-49`):

Container	Partition Key	Inhalt
<code>users</code>	<code>/id (= uid)</code>	Profil + verschlüsselte LinkedIn-Verbindung (Feld <code>linkedin</code>)
<code>sessions</code>	<code>/id (= sessionId)</code>	Session-Metadaten + Ownership (<code>uid</code>)
<code>runs</code>	<code>/sessionId</code>	Analyse-Läufe (1 Session → N Runs)
<code>usage</code>	<code>/id (= uid_YYYY-MM-DD)</code>	Tages-Token-Budget, container-seitige TTL 35 Tage

Kern-Interfaces (`src/lib/server/runs-store.ts:23-46`):

```

interface SessionDoc { id: string; uid: string; updatedAt: string /* ISO */ }

interface RunDoc {
  id: string; // UUID (crypto.randomUUID)
  sessionId: string; // = Partition Key
  uid: string;
  createdAt: string; // ISO-String – Cosmos hat KEINEN Timestamp-Typ!
  conversationType: string;
  conversationSubType: string | null;
  goal: string | null;
  lang: 'de' | 'en' | null;
  jurisdiction: string | null;
  transcriptText: string | null; // nur wenn storeTranscript === true
  analysisJson: {
    summary; strengths[]; improvements[]; rewrites[]; riskFlags[];
    practice7Days?; scores; competency_ratings[]; competency_error?; quality_notes[];
  };
  ragContext: { cards[]; count; error? };
  summary: string | null;
  scoreOverall: number | null;
  rating?: number; // 1-5
  ratedAt?: string; // ISO
}

type UserProfileDoc = {
  id: string; // = uid (Partition Key)
  email: string; displayName: string;
  language?: Locale; createdAt: string; updatedAt: string;
  // linkedin?: StoredConnection – verschlüsselt, NIE in publicProfile() zurückgegeben
};

type UsageDoc = { id: string; uid: string; date: string; tokensUsed: number; updatedAt: string

```

`UserProfileDoc` wird beim ersten `GET /api/users/profile` **automatisch provisioniert** — ein bewusst offenes Produkt ohne Einladungs-Flow (`src/app/api/users/profile/route.ts:49-63`).

CRUD-Helfer (`src/lib/cosmos.ts:50-96`) — dünne, generische SDK-Wrapper:

```

readItem<T>(container, id, partitionKey): Promise<T | null> // 404 → null
upsertItem<T>(container, item): Promise<T> // Voll-Upsert, KEIN Feld-Merge
queryItems<T>(container, sql, params: SqlParameter[]): Promise<T[]>
deleteItem(container, id, partitionKey): Promise<void> // 404 → no-op

```

Es gibt **keine Transaktionen** und **keinen Batch-Operator**.

6. Persistenz-Muster & Cosmos-Eigenheiten

Aus der „kein Merge / keine Transaktion“-Beschränkung folgen drei wiederkehrende Muster, die der Reviewer kennen muss:

(a) Read-Modify-Upsert (Rating, Profil-PATCH, LinkedIn-Token-Speicherung): Das gesamte Dokument wird gelesen, im Speicher verändert und neu geschrieben (`rateRun` , `src/lib/server/runs-store.ts:157-171`). Konsequenz: „**last write wins**“, keine optimistische Nebenläufigkeit (kein etag-Check). Bei der erwarteten Last (ein Nutzer bewertet seinen eigenen Run) unkritisch, aber nicht race-safe.

(b) Atomarer Patch- `incr` (nur Token-Budget, `src/lib/server/cost-cap.ts:86-89`): Cosmos' server-seitige `patch([op: 'incr', path: '/tokensUsed', value])` -Operation ist atomar und damit race-sicher gegen parallele Analyse-Calls. Bei 404 (Doc des Tages existiert noch nicht) wird es angelegt; ein 409 (Race beim Anlegen) löst einen `Retry- incr` aus.

(c) Cursor-Pagination über `createdAt` (`src/lib/server/runs-store.ts:93-155`): Die Run-Liste nutzt `SELECT TOP {limit+1} ... WHERE c.sessionId=@sid AND c.createdAt < @cursor ORDER BY createdAt DESC` . Der `cursor` (eine runId) wird via `getRun` zu seinem `createdAt` aufgelöst. Die Listen-Query **projiziert ohne** `transcriptText` / `analysisJson` -Vollkörper (RU- und Payload-Ersparnis).

⚠ **Befund:** `createdAt` ist ein ISO-String. Bei zwei Runs mit identischem `createdAt` könnte die Cursor-Pagination Einträge duplizieren oder überspringen (kein `id` -Tiebreaker). In der Praxis sind die Zeitstempel ms-genau und damit faktisch eindeutig, aber als Korrektheits-Annahme zu dokumentieren.

Ownership (`checkSessionOwnership` , `src/lib/server/runs-store.ts:52-59`): liest das Session-Dokument; gehört es einer fremden uid → `{allowed:false}` (403); existiert es nicht → `{allowed:true}` (wird beim Save angelegt). Wird **immer vor** jedem Write/Read aufgerufen.

7. Authentifizierung & Session

NextAuth v5 + Microsoft Entra ID (`src/auth.ts:13-34`):

- Single Provider `microsoft-entra-id`, Issuer `https://login.microsoftonline.com/{TENANT}/v2.0` (Tenant `common`), Session-Strategie **JWT im HTTP-only-Cookie** (kein Bearer-Token), `trustHost: true` (Pflicht hinter dem App-Service-Proxy).
- **uid-Quelle:** der `jwt` -Callback schreibt `profile.sub` (Microsoft OIDC subject) in `token.uid`, der `session` -Callback in `session.user.id` (`src/auth.ts:24-34`). Diese `uid` ist der Partition-Key des `users` -Containers und das Ownership-Kriterium der gesamten App.
- **Audience** `AzureADandPersonalMicrosoftAccount` (in der App-Registrierung) — erlaubt **persönliche** Microsoft-Konten (@outlook.de) zusätzlich zu Organisations-Konten. Falsch gesetzt würden persönliche Logins fehlschlagen (`docs/azure-migration/PLAN.md:81-82`).

Server-Vertrag (`src/lib/api-auth.ts:13-41`):

```
verifyAuthToken(req): Promise<{ uid, email } | null> // liest auth()-Session, extrahiert use
requireAuth(req): Promise<{ uid, email } | NextResponse> // 401-Response bei fehlender Sessio
```


Die Vertragsform `{ uid, email } | 401` ist bewusst **identisch** zum früheren Firebase-Helfer → die ~15 Routen blieben bei der Migration nahezu unverändert (Leitprinzip „Vertragstreue“).

Client (`src/providers/auth-provider.tsx:61-154`): `AuthProvider` liest `useSession()` und leitet einen schlanken `AuthUser`-Shim ab (`uid, email, displayName, photoURL`). **Referenzielle Stabilität ist kritisch** — alle Funktionen in `useCallback`, der Context-Value in `useMemo`. Ohne diese Disziplin entstünde in den vielen `authFetch`-Konsumenten eine Endlos-Refetch-Schleife (hart erkaufte Lektion aus der Schwester-App). Nach Login holt der Provider das frische Profil aus `/api/users/profile` (löst auch die Provisionierung aus).

authFetch (`src/lib/api-client.ts:11-32`): setzt `credentials: 'same-origin'` (Browser sendet das Session-Cookie automatisch), Header `Content-Type: application/json + x-locale` — **kein** Bearer-Token (Cookie-Auth).

Schutz & offene Flächen: `AuthGuard` (`src/components/auth/auth-guard.tsx:17-53`) wrappt geschützte Seiten; unauthentifiziert → `LoginModal` (Microsoft-Sign-In via `signInWithMicrosoft()`, `src/lib/auth-service.ts:10-26`). / → Redirect auf `/analyze`. Die **Middleware ist bewusst auth-agnostisch** (`src/middleware.ts:1-46`): sie setzt ausschließlich den `NEXT_LOCALE`-Cookie (Accept-Language-Heuristik) und prüft **keine** Auth. Der Schutz liegt konsequent in `AuthGuard` (Client) + `requireAuth` (Server).

8. API-Surface (~15 Routen) + Fehler-/Statusmodell

Jede Route ist ein App-Router-Handler mit **Standard-Envelope** `{ ok: boolean, error?, code?, ... }`.

Routen-Übersicht:

Route	Methode	Zweck	Auth	Rate-Limit	Budget
/api/auth/[... nextauth]	GET/POST	NextAuth-Handler	—	—	—
/api/analyze	POST	Feedback (RAG) + Kompetenz-Scoring	✓	10/min/IP	✓
/api/competencies	POST	Nur Kompetenz-Scoring (Solo, lazy import)	✓	10/min	✓
/api/runs/save	POST	Run persistieren (Ownership vor Write)	✓	20/min	—
/api/runs/list	GET	Run-Historie (Cursor-Pagination)	✓	30/min	—
/api/runs/get	GET	Einzel-Run (IDOR-sicher)	✓	30/min	—
/api/runs/rate	POST	1–5-Sterne-Rating	✓	20/min	—
/api/users/profile	GET/PATCH	Profil lesen/ändern, Auto-Provisionierung	✓	30 / 20/min	—
/api/linkedin/auth	POST	OAuth-Start (HMAC-signierter state)	✓	—	—
/api/linkedin/callback	GET	OAuth-Callback (state-Verifikation)	— (state)	—	—
/api/linkedin/status	GET	Verbindungs-Status	optional	—	—
/api/linkedin/post	POST	Posten (Bestätigung im UI)	✓	5/min	—
/api/linkedin/generate-post	POST	Post-Text (Gemini)	✓	5/min	—
/api/linkedin/generate-image	POST	Post-Bild (NanoBanana)	✓	3/min	—
/api/pinecone-smoke	GET	RAG-Diagnose	✓	10/min	—

Fehler-/Statusmodell (durchgängig, z. B. `src/app/api/runs/save/route.ts:133-233`):

HTTP	Code	Auslöser
400	(Zod-FlattenedError) / <code>BAD_SESSION_ID</code> / <code>BAD_CURSOR</code>	Eingabe-/Cursor-Validierung
401	<code>UNAUTHORIZED</code>	keine gültige Session
403	<code>FORBIDDEN</code>	Ownership verletzt
404	<code>NOT_FOUND</code>	Run/Profil nicht gefunden
429	<code>RATE_LIMITED</code> (IP) / <code>QUOTA_EXCEEDED</code> (Budget)	Drosselung
500	<code>INTERNAL_ERROR</code>	unerwartet (geloggt)

Gate-Reihenfolge (verifizierbar in `route.ts`): `requireAuth` (401) läuft **vor** dem Zod-Parse (400) — Auth-Fehler haben Vorrang vor Validierungsfehlern. Bei den teuren Routen folgt zwischen Auth und Flow der Cost-Cap (429). Fehlermeldungen sind lokalisiert (`getApiMessages(req)` → `x-locale`-Header → `NEXT_LOCALE`-Cookie → Default, `src/lib/server/get-request-locale.ts:13-38`).

Beispiel-Response `/api/analyze` (`src/app/api/analyze/route.ts`):

```
{ "ok": true, "result": {
  "summary": "...", "strengths": [...], "improvements": [...], "rewrites": [...], "riskFlags": [
  "scores": { "overall": 7 },
  "rag_context_cards": [ ... ], "rag_context_count": 8, "rag_error": null,
  "competency_ratings": [{ "id": "C1", "name": "...", "evidence": [...], "why": "...", "score": 3, "confiden
  "competency_error": null,
  "quality_notes": [{ "code": "EVIDENCE_NOT_GROUNDED", "severity": "warn", "message": "...", "field": "
}}
```

9. Die KI-Schicht — Flows & Prompt-Kette

Genkit wird mit dem `googleAI()` -Plugin und Modell `googleai/gemini-2.5-flash` initialisiert (`src/ai/genkit.ts:1-7`). Drei Flows arbeiten zusammen:

Flow 1 — `generateDynamicFeedback` (`src/ai/flows/generate-dynamic-feedback.ts:13-122`): ein **orchestrierender Wrapper** (kein `defineFlow`). Ablauf:

1. `buildRetrievalQuery()` verbindet `conversationType` + `subType` + `goal` + Transkript (auf 4000 Zeichen gekürzt).
2. `buildBaseFilter()` erstellt Filter für `conversation_type` / `jurisdiction`.
3. `pineconeSearchCards({ text, topK:8, filter, lang })` — bei 0 Treffern mit `lang` -Filter folgt ein zweiter Aufruf **ohne** `lang` -Filter (`:82-85`).
4. `cardsToSnippets()` formatiert max. 8 Treffer als `[#id score=x.xxx]\nchunk` (je 1800 Zeichen).
5. Delegation an `generateTailoredFeedback({ ..., relevantSnippets })`.
6. Anreicherung des Ergebnisses um `rag_context_cards`, `rag_context_count`, `rag_error`. Ein Retrieval-Fehler wird **nicht geworfen**, sondern als `rag_error` zurückgegeben.

Flow 2 — `generateTailoredFeedback` (`src/ai/flows/generate-tailored-feedback.ts:9-102`): der eigentliche Genkit-Flow (`definePrompt` + `defineFlow`). Input/Output siehe Anhang B. Das Handlebars-Template (verbatim in Anhang B) fokussiert das Modell auf die **Führungskraft**, verbietet echte Namen + das Offenlegen der Wissensbasis, steuert die Ausgabesprache über `{{lang}}` und bindet die RAG-Snippets nur als „guidance“ ein. Der `inputText` wird durch `sanitizeForPrompt(..., {label: 'TRANSCRIPT'})` gehärtet (Fencing + Injection-Logging).

Flow 3 — `scoreCompetencies` (`src/ai/flows/score-competencies.ts:10-121`): bewertet die zehn Kompetenzen entlang des C1–C10-Modells. Details zum Schema-Forced Reasoning in Abschnitt 10, der verbatim-Prompt in Anhang B.

Orchestrierung in `/api/analyze` (`src/app/api/analyze/route.ts:105-208`): beide Flows laufen **parallel** via `Promise.allSettled`. `generateDynamicFeedback` ist Pflicht (`Reject` → `throw`), `scoreCompetencies` optional (`Reject` → `competency_error` + `defaultCompetencyRatings()`). Danach:

- **Label-Substitution** in Evidenz-Zitaten (`leader/employee` → „Führungskraft“/„Mitarbeiter:in“, `:152-157`),
- `normalizeScore` (nur 1–4, sonst `null`, `:51-56`), `normalizeEvidence` (max. 2 Items, `:62-65`),

- sprach-abhängiger Fallback-Text für nicht beobachtbare Kompetenzen („not sufficiently observable“ / „nicht ausreichend beobachtbar“).

`/api/competencies` (`src/app/api/competencies/route.ts:48-63`) ist ein eigenständiger Endpunkt für das Scoring; es nutzt **Lazy Import** von `scoreCompetencies` (Crash-Isolation beim Modul-Load).

10. Schema-Forced Reasoning & Prompt-Design

Der didaktische Differenzierer von `scoreCompetencies` ist **Schema-Forced Reasoning**: das Output-Schema (`CompetencyRatingSchema` , `src/ai/flows/score-competencies.ts:17-48`) ordnet die Felder bewusst **evidence** → **why** → **score** — das LLM füllt sie in dieser Reihenfolge und wird so gezwungen, **erst zu belegen, dann zu begründen, erst dann zu urteilen**. Score-vor-Evidenz verleitet das Modell dazu, zuerst zu urteilen und danach zu rationalisieren („rationalization bias“).

Drei Prompt-Regeln verstärken das (verbatim-Auszug Anhang B):

- **SPRACHE** (:54-60): `why` in der Transkript-Sprache (`{{lang}}`), Zitate im Originalwortlaut.
- **REIHENFOLGE** (:72-76): „1) Sammle EVIDENCE, 2) Begründe nur daraus, 3) Vergib ERST DANN score. WENN NICHT ERKENNBAR: `evidence=[]`, `score=null`. Erfinde KEINE Zitate.“
- **KEIN ÜBER-SCORING** (:83-87): „Reine Terminabsprachen/Logistik/Small-Talk sind KEINE Führungsleistung → `score=null`.“

Die `.describe()`-Felder im Schema sind **load-bearing** (sie tragen die Semantik ins Structured-Output-Constraint). Die Skala ist `z.number().min(1).max(4).nullable()`; `confidence` ist `z.number().min(0).max(1).nullable().optional()` (Achtung: `optional` ≠ `nullable` — Empfänger-Code muss defensiv `typeof` ≡ `'number'` prüfen).

Bewertungs-Hinweis: Schema-Forced Reasoning ist eine Prompt-/Schema-Konvention, keine technische Garantie. Gemini 2.5 Flash *folgt* der Feldordnung üblicherweise, aber der Reviewer sollte über den Quality-Harness (`--n=3`) prüfen, ob Scores tatsächlich evidenzgestützt sind (siehe Abschnitt 14 — der Harness fand hier reale Über-Scoring-Fälle, die per Prompt gefixt wurden).

Beide produktiven Flows nutzen Genkit-**Handlebars-Templates** mit bedingten Blöcken (`{{#if lang}}` , `{{#each relevantSnippets}}`) und Triple-Stash (`{{{...}}}` , kein HTML-Escaping) — damit ist der Prompt byte-identisch zur Vorversion, wenn ein optionales Feld fehlt.

11. RAG / Pinecone-Retrieval

Konfiguration (`src/lib/pinecone.ts:1-40,174-216`): `getPineconeConfig()` liest `PINECONE_API_KEY` , `PINECONE_INDEX_HOST` (bereinigt um `https://` und `/`), `PINECONE_NAMESPACE` (Default `__default__`). Endpunkt: POST `https://{host}/records/namespaces/{ns}/search` , Header `Api-Key` + `X-Pinecone-API-Version: 2025-10` . Genutzt wird die „**Search with text**“-API (Integrated Embedding — Text-Input, kein clientseitiges Embedding), Request-Body `{ query: { inputs: {text}, top_k, filter?, fields }` . Die

Antwort wird durch `toCompat()` in `{ raw, hits, matches, results, count }` umgeformt (`{id, score, metadata}` pro Treffer).

`normalizeArgs()` akzeptiert mehrere Aufruf-Signaturen (String oder Objekt mit `text/topK/lang/filter`); `DEFAULT_FIELDS` umfasst 28 Metadatenfelder (`chunk_text, title, conversation_type, competency_ids, lang, jurisdiction, ...`).

Anti-Halluzination: Der Feedback-Prompt nutzt die Snippets nur als interne „guidance“ und darf die Vektor-DB nicht offenlegen; der deterministische Quality-Layer (Abschnitt 14) prüft zusätzlich, dass jede Kompetenz-Evidenz **wörtlich im Transkript** steht.

`/api/pinecone-smoke` (`src/app/api/pinecone-smoke/route.ts:8-39`) ist ein authentifizierter Diagnose-Endpunkt (`?text=...&lang=...&topK=...`), der rohe Treffer + Count zurückgibt.

⚠ **Befund:** `buildRetrievalQuery` kürzt das Transkript auf **4000 Zeichen** — bei langen Gesprächen geht ein Großteil des Kontexts für die Retrieval-Query verloren (die Analyse selbst sieht aber das volle, gefenkte Transkript bis 500k). Der RAG- `lang` -Fallback (zweite Suche ohne Sprachfilter) kann semantisch andere Karten zurückbringen.

12. Kompetenzmodell C1–C10

Das maßgebliche Modell für die **API-Antwort** (`src/app/api/analyze/route.ts:27-38`):

ID	Name (API-Route)	Fokus
C1	Integrieren und Verbinden	Beziehungen, Teamzusammenhalt
C2	Klarheit und Entscheidungsstärke	klare Ansagen, Entscheidungen
C3	Befähigen und Entwickeln	Coaching, Delegation, Wachstum
C4	Sicherheit und Stabilität geben	psychologische Sicherheit
C5	Kommunikation und Kooperation	Zuhören, Dialog
C6	Zielorientierte Umsetzung	Fokus, Verbindlichkeit
C7	Innovative Kultur fördern	Offenheit, Veränderung
C8	Selbstreflexion und Lernmotivation	Reflexion, Feedback-Akzeptanz
C9	Zukunftsorientierung & strategischer Weitblick	Vision, Strategie
C10	KI- und Datenkompetenz	datengetriebene Führung

Skala (`src/ai/flows/score-competencies.ts:41-46`): 1 = schwach/kontraproduktiv, 2 = solide Ansätze, 3 = gut/überwiegend wirksam, 4 = sehr gut/vorbildlich; `null` = nicht beobachtbar. Pro Kompetenz: `evidence: string[]` (max. 3, anonymisiert), `why` (Begründung) und optional `confidence` (0–1).

⚠ **Befund (kritisch):** Es existiert ein **zweites, divergentes** Kompetenz-Modell in `src/lib/competencies.ts:6-17` mit abweichenden Namen bei **C2, C4, C7, C9, C10** — z. B. C2 =

„Inspirieren und Aktivieren“, C4 = „Kundenorientierung“, C7 = „Gestaltung des Wandels“, C9 = „Strategische AI Literacy“, C10 = „Entscheiden, Steuern und Delegieren in Human-AI Hybrid Teams“. Die API-Antwort nutzt die Route-Version, ein UI-/Profil-Layer potenziell die `lib`-Version → die angezeigten Labels können auseinanderlaufen. Zu klären: welches Modell ist autoritativ, und sollen beide synchronisiert werden? (Siehe Abschnitt 20/21.)

13. Kosten- & Missbrauchsschutz

Drei Schichten, in `/api/analyze` und `/api/competencies` in dieser Reihenfolge verdrahtet: **requireAuth** → **IP-Rate-Limit** → **Cost-Cap** → **Flow**.

(a) Cost-Cap — Pro-User-Token-Budget/Tag (`src/lib/server/cost-cap.ts`): der eigentliche Kostenschutz. Algorithmus von `checkAndConsumeBudget({uid, email, estimatedTokens})`:

1. Admin-Bypass: ist `email` in `ADMIN_EMAILS` -Allowlist → `allowed:true` (Verbrauch wird dennoch getrackt).
2. `todayKey(uid) = ${uid}_${YYYY-MM-DD}` (UTC, instanzstabil).
3. Atomarer `patch({op:'incr', path:'/tokensUsed', value: inc}, {op:'set', path:'/updatedAt', ...})`.
4. Bei 404: `items.create({..., tokensUsed: inc})`; bei 409-Race: erneuter `incr`.
5. Bei sonstigem Cosmos-Fehler: **fail-open** (`allowed:true`) — die Budget-Infra darf den eigentlichen Request nicht killen.
6. `mode==='enforce'` und `used > limit` → **429** `QUOTA_EXCEEDED`.

ENV: `DAILY_TOKEN_LIMIT` (Default 500 000), `COST_CAP_MODE` (`enforce / off`), `ADMIN_EMAILS`.
`estimateTokens = chars/4 + 3000` (grobe Vorab-Reservierung, **kein** Retro-Accounting, :135–140). Der Zähler lebt in Cosmos und gilt **instanzübergreifend**.

(b) Rate-Limit — In-Memory, pro IP (`src/lib/rate-limit.ts:5–77`): `Map<key, {count, resetAt}>`, Key `prefix:ip` (erstes Octet aus `x-forwarded-for`), lazy Cleanup alle 5 min. Liefert `null` (erlaubt) oder 429 (`RATE_LIMITED` + `Retry-After`). **Bewusste Schwäche**: prozess-lokal, **nicht** über App-Service-Instanzen koordiniert → auf Multi-Instance effektiv `N × Limit` umgehbar. Dient als Nuisance-Schutz; der harte Kostenschutz ist (a).

(c) Prompt-Guard (`src/lib/prompt-guard.ts:14–93`): 14 Regex-Muster für Injection („ignore/disregard/forget previous instructions“, „you are now a“, „new system prompt“, „act as“, „DAN/jailbreak“, `[SYSTEM]` / `[INST]` / `<<SYS>>` / `<im_start>`, „reveal/output your system prompt“). `sanitizeForPrompt` **erkennt** (`detectInjection`), **fenced** (`fenceUserContent` mit XML-Delimitern + Reminder) und **kürzt** auf 500k — **blockt aber nicht**. Detection ist Telemetrie (`console.warn`); der Flow läuft mit gefenctem Input weiter. Eingebunden in beide Feedback-Flows und das Scoring.

14. Qualitäts-Layer

(a) Laufzeit — deterministische Checks (`src/lib/quality-core.ts`, `src/lib/server/quality-checks.ts`): reine Funktionen (kein I/O), ENV-gegated (`QUALITY_CHECKS_MODE` = `off|warn|enforce`, Default `warn`), liefern `QualityNote[]` (`{code, severity: info|warn|error, message, field?}`) und **werfen nie**:

Check	Code	Logik
<code>checkEvidenceGrounding</code>	<code>EVIDENCE_NOT_GROUNDED</code>	Evidenz-Zitat (nach <code>evidenceNeedle</code> - Normalisierung) muss wörtlich im anonymisierten Transkript stehen
<code>checkScoreHasEvidence</code>	<code>SCORE_WITHOUT_EVIDENCE</code>	Score gesetzt, aber <code>evidence</code> leer
<code>checkRewritesDiffer</code>	<code>REWRITE_NOT_DIFFERENT</code>	<code>better</code> $\equiv$ <code>original</code>
<code>checkBannedPhrases</code>	<code>BANNED_PHRASE</code> (info)	Floskel-Hardlist im Summary

`evidenceNeedle` (`src/lib/quality-core.ts:27-35`) strippt einen beliebigen führenden Sprecher-Prefix (`^\p{L}[\p{L}\d _-]{0,24}`) + Anführungszeichen und normalisiert Whitespace/Case; Zitate `< 8` Zeichen werden nicht geprüft. Verdrahtet in `/api/analyze` (`:175-204`); die Notes landen sichtbar im Response (`quality_notes`), werden persistiert (`analysisJson.quality_notes`) und im Report als Banner gezeigt (`src/components/app/report-dashboard.tsx:285-306`).

Hinweis: `mode=enforce` blockt nur bei `severity:error` — da aktuell kein Check `error` erzeugt, ist **enforce faktisch $\approx$ warn**. Bewusster Expansion-Point (z. B. Grounding zu `error` hochstufen), aber nicht offensichtlich.

(b) Offline — Regression-Harness + Golden-Set (`scripts/quality/run-regression.ts`, `golden-set.json`): misst den **echten** `scoreCompetencies` -Flow (Quellen direkt injiziert $\rightarrow$ saubere Isolation, kein RAG/keine Persistenz) gegen 5 Szenarien (3 DE, 2 EN), darunter explizite Negativ-Tests (z. B. „03-zu-kurz“ = alle Kompetenzen müssen `null` sein). Jedes Szenario: `{ transcript, expect:{ observableCompetencies, notObservableCompetencies, mustNotMention, expectedLang } }`.

- **Harte** Checks (FAIL): Faithfulness/Grounding, Negativ-Test (nicht beobachtbar $\Rightarrow$ `score=null`), Halluzination (verbotene Begriffe nicht in `why` / `evidence`).
- **Weiche** Checks (WARN): Observable (beobachtbar $\Rightarrow$ `score` vorhanden), Locale (Begründungen der gescorten Kompetenzen in der Zielsprache), Stabilität (Score-Spannweite bei `--n>1`).

Exit-Code 0/1 (CI-tauglich). Flags `--only=<id>`, `--n=<k>`, `--judge`. Reproduzierbar via `npm run quality:regression`.

(c) LLM-as-Judge — neutraler Fremd-Judge (`scripts/quality/judge.ts`): **Claude** (Modell `claude-opus-4-...`, env `JUDGE_MODEL`) — bewusst **nicht** „Gemini bewertet Gemini“. 6 Dimensionen je 1–5, **n=3 mit Median**: `faithfulness`, `coverage`, `actionability`, `competency_consistency`, `tone`, `locale`. Gegated auf `ANTHROPIC_API_KEY` (ohne Key: stiller Skip, nur deterministische Checks). Ein `--judge` -Lauf kostet ~ 18 Claude-Calls (3 $\times$ 6 Dim) $\approx$ \$0.3. Judge-Prompt + Rubrik sind hardcoded mit dokumentiertem „SLOT“ zum Ersetzen durch die erprobte Schwester-App-Rubrik.

Bewährter Zyklus (docs/quality/README.md): Der Loop fand beim ersten Lauf zwei echte Generator-Befunde (deutsche Begründungen bei EN-Input; Über-Scoring trivialer Transkripte), beide wurden am Generator gefixt und grün re-gemessen (**5/5 bei n=3**). Owner-Regel: **jeder neue Gate braucht einen Negativ-Test** im Golden-Set.

15. Frontend (App Router, Client-Komponenten, Transcript-Tooling)

Routen: / → Redirect /analyze ; /analyze (AuthGuard + AnalyzeClient , force-dynamic); /runs-dashboard (Historie); /runs/{sessionId}/{runId} (Report); /design-preview (in Produktion via notFound() gesperrt, src/app/design-preview/page.tsx:12). Das Root-Layout ist async , löst die Locale aus dem NEXT_LOCALE -Cookie auf und setzt <html lang={localeBcp47[locale]}> (src/app/layout.tsx:23-36).

AnalyzeClient (src/app/analyze/AnalyzeClient.tsx): die komplexeste Client-Komponente (~31 useState). Funktionen: lokales PDF-Parsing, Teams-Cleanup, Sprecher-Erkennung + Auto-Zuordnung von leaderLabel / employeeLabel , Anonymisierung mit Live-Preview (auf 400 Zeichen gekürzt), Privacy-Mode (default-on), Upload-Modus replace / append , Undo. **Doppel-Submit-Schutz** via busyRef (synchron, gegen schnelle Doppelklicks, :70,163-214). **Save-Retry** (pendingSave): scheitert nur die Persistenz, bleibt das (bezahlte) LLM-Ergebnis im State und kann ohne erneuten LLM-Call gespeichert werden (:234-249,554-561).

ReportDashboard (src/components/app/report-dashboard.tsx:184-545):

- **ScoreRing** (src/components/app/score-ring.tsx:9-38): SVG-Kreis 0–100 % (strokeDashoffset - Animation, drop-shadow), Zentral-Wert oder „—“.
- **InsightCard** (src/components/app/insight-card.tsx): Ton-System success/warning/danger (emerald/amber/red) für Stärken/Verbesserungen/Risiken.
- **CompetencyPanel** (:28-112): C1–C10 als ausklappbare <details> , Score-Skalierung (≤4 → /4, ≤10 → /10, sonst clamp) → Balken-Ton, Zitate (max. 3).
- **RatingCard** (:119-177): 5 Sterne mit Hover-Preview → POST /api/runs/rate , State-Machine idle/saving/saved/error.
- **JSON-Download** (gesamtes result), **7-Tage-ICS-Erinnerung** (handleReminder , RRULE FREQ=DAILY;COUNT=7 , CRLF-Zeilenenden, Escaping), **Transkript-Ansicht** (conditional), **Degradations-Banner** (rag/competency/grounding, :285-306).
- **LinkedInPostCard** (Abschnitt 17).

RunsDashboardClient : Cursor-Pagination (Limit 50, „Mehr laden“, Dedup über Set), Suche + Sortierung in useMemo (useDeferredValue , Sort date_* / score_*).

Hilfsmodule (reine Funktionen, testbar): src/lib/transcript-utils.ts (Cleanup/Detect/Sanitize), src/lib/report-utils.ts (unwrapRunResult , overallToPercent , scoreTitle/Badge , parseRewrite , pickPractice), src/lib/pdf/parsePdfToText.ts , src/lib/session-utils.ts (newSessionId via crypto.randomUUID , shortId).

⚠ **Befunde:** Das Dashboard formatiert Datumswerte hart mit `de-DE` (ignoriert die aktive Locale); die ICS-Erinnerung setzt immer 9:00 **UTC** (Zeitzone des Nutzers unberücksichtigt); `ReportDashboard` fällt von `result.competency_ratings` auf `result.competencies` zurück (Feldnamen-Drift möglich).

16. Internationalisierung (7 Sprachen)

7 Locales (`src/i18n/config.ts:1-24`): `en`, `de`, `fr`, `it`, `es`, `pl`, `cs`, Default `en`, **DE als Source-of-Truth**. BCP-47-Map für `<html lang>`. Einziger Cookie: `NEXT_LOCALE` (der frühere Firebase- `__session` -Fallback wurde bei der Migration entfernt, da auf Azure obsolet und schädlich — er hätte `NEXT_LOCALE` überschrieben). `getDictionary(locale)` mit Fallback `en` (`src/i18n/dictionaries/index.ts`). `useTranslation` liest die Locale aus dem `AuthProvider`. Der `LanguageSwitcher` (`src/components/language-switcher.tsx:14-49`) delegiert an `updateLanguage` → schreibt Cookie **und** (falls eingeloggt) das Cosmos-Profil. Server-seitige Fehlermeldungen werden über `getApiMessages` lokalisiert. Die Dictionary-Key-Parität über alle 7 Sprachen wird bei Erweiterungen per Skript geprüft (zuletzt 172/172 Keys).

17. LinkedIn-Integration

Optionales Feature, das aus einer Analyse einen LinkedIn-Post erzeugt und veröffentlicht.

OAuth-Flow (`src/app/api/linkedin/auth/route.ts`, `callback/route.ts`, `src/lib/server/linkedin-connection.ts:74-109`):

- Start als **POST mit App-Auth** (nicht GET-Navigation). Der Server erzeugt einen **HMAC-SHA256-signierten state**, der die **uid trägt** (`createSignedState`, Payload `base64url({uid, nonce, exp: now+10min})` + Signatur), plus ein `returnTo` -Cookie (Open-Redirect-geschützt: nur app-interne Pfade).
- Der Callback prüft `state` **doppelt** (Cookie-Gleichheit für CSRF **und** Signatur via `timingSafeEqual`), tauscht Code→Token→Profil (`sub` = `personUrn`) und speichert die Verbindung **pro User in Cosmos** (`users.linkedin`).

Token-Verschlüsselung (`src/lib/server/linkedin-connection.ts:26-68`): AES-256-GCM, Format `v1.{iv_b64url}.{tag_b64url}.{ciphertext_b64url}`, Schlüssel aus `LINKEDIN_TOKEN_KEY` (empfohlen) oder abgeleitet aus `LINKEDIN_CLIENT_SECRET` (SHA-256). `decryptToken` prüft den AEAD-Tag; bei Schlüssel-Rotation oder korrupten Daten → `null` (= „nicht verbunden“, sauberer Reconnect statt Crash). `getLinkedInConnection` prüft Ablauf (`Date.now() ≥ expiresAt`).

Status & Posten: `/api/linkedin/status` liefert `{configured, connected, expired, name}` (die Card zeigt damit „nicht konfiguriert“ statt auf eine rohe Fehlerseite zu navigieren). `generate-post` (Gemini, mehrsprachig, optionaler `LINKEDIN_BRAND_NAME`), `generate-image` (NanoBanana, 3/min — teuerster Endpunkt), `post` (Bestätigungs-Dialog vor dem **PUBLIC**-Posten, `tokenExpired` -Handling → 401, Client fordert Reconnect). Alle drei mit `requireAuth` + Rate-Limit.

Offen: echte LinkedIn-Credentials + Referenzbilder (`public/linkedin/`) sind noch Platzhalter; kein Token-Refresh-Flow (LinkedIn-Token läuft ab → Reconnect).

18. Sicherheitsmodell

IDOR-Fix (zentral): Run-Detailseiten luden früher serverseitig per Admin-SDK **ohne** Auth-Check — jeder mit der URL konnte fremde Transkripte lesen. Jetzt ist `page.tsx` nur ein `AuthGuard`-Wrapper, und `RunDetailClient` lädt den Run clientseitig über `GET /api/runs/get`, das `requireAuth` **und** `checkSessionOwnership` erzwingt (`src/app/runs/[sessionId]/[runId]/page.tsx:6-25`, `RunDetailClient.tsx:107-133`).

Ownership vor jedem Zugriff: `checkSessionOwnership` läuft in `save` **vor** dem Write (keine Session-Übernahme durch uid-Überschreiben), ebenso vor `get / list / rate` (Abschnitt 8).

Secrets: alle in Azure Key Vault (`coach-kv-20f84`, 7 Secrets: AUTH-SECRET, ENTRA-CLIENT-SECRET, COSMOS-KEY, GEMINI-API-KEY, PINECONE-API-KEY, LINKEDIN-CLIENT-ID/-SECRET), als App-Setting-Referenzen aufgelöst. `.env*` ist gitignored (`!.env.example`); SA-Keys (`workspace/*.json`) gitignored.

Kryptografie: LinkedIn-Token AES-256-GCM (AEAD mit Auth-Tag); OAuth- `state` HMAC-SHA256 mit `timingSafeEqual`.

Transport: HTTP-only-Session-Cookie, `SameSite=lax`; HTTPS via App Service. Prompt-Injection- Härtung (Fencing) in allen Flows.

Bewusste Designentscheidungen, die der Reviewer kennen muss: offene Selbst-Provisionierung (kein Einladungs-Flow); Prompt-Guard loggt statt blockt; Rate-Limit nicht verteilt; CSRF stützt sich auf `SameSite` + NextAuth (kein expliziter CSRF-Token auf den Custom-POST-Routen außer dem LinkedIn- `state`); Cost-Cap fail-open bei Cosmos-Fehlern.

19. Infrastruktur & Deployment (Azure)

Ressourcen (geteilte „PulseCraft“-Basis, West Europe):

- Web-App `pulsecraft-coach` (Node 22, B2-Linux-Plan), `output: 'standalone'`, App-Settings `WEBSITE_RUN_FROM_PACKAGE=1`, `ENABLE_ORYX_BUILD=false`, `PORT=8080`, Startup-Command `node server.js`.
- **Cosmos DB** `pulsecraft-prod-cosmos` → Datenbank `coach` (Container `users/sessions/runs/usage`).
- **Key Vault** `coach-kv-20f84` + System-Managed-Identity (Get/List), App-Settings als KV-Referenzen (Status „Resolved“).
- **Entra-App** `2b24d264-1643-4b12-846b-f1a9e1554e9f` (Audience `AzureADandPersonalMicrosoftAccount`, Redirect-URLs `prod` + `http://localhost:9002`).

Belege: `docs/azure-migration/PLAN.md:19,36-37,81-84`, `next.config.ts`.

Deploy-Pipeline: manuell (kein CI/CD). Ablauf: `next build` → `static/` + `public/` ins `.next/standalone/` kopieren (Pflicht beim Standalone-ZIP, sonst fehlen Assets) → ZIP → `az webapp deploy ... --type zip`. Runbook in Anhang C.

DSGVO-Stand: Auth, Daten, Hosting, Secrets in der EU (Microsoft-AVV). Gemini + Pinecone noch außerhalb EU → Folgeschritt **M7** (Empfehlung: Gemini via Vertex AI EU für Qualitätserhalt statt Wechsel zu Azure OpenAI; Pinecone → Azure AI Search unter Wiederverwendung der Embeddings).

Firebase-Rückbau verifiziert: 0 `firebase / firestore` -Importe in `src/`; `firebase-admin` nur noch transitiv über `@genkit-ai/firebase` (ungenutzt, bleibt wegen Genkit/Gemini). Config-Dateien (`.firebaserc`, `firebase.json`, `firestore.rules`, `apphosting.yaml`) entfernt; GCP-Backend deaktiviert; Firestore-Daten als 30-Tage-Backup belassen.

20. Bekannte Schwächen, Risiken & Roadmap

Konsolidiert aus dem Code-Audit, priorisiert.

Hoch (vor breiterem Rollout adressieren):

1. **Kompetenz-Modell-Divergenz** (`route.ts:27-38` vs. `lib/competencies.ts:6-17`): abweichende Namen bei C2/C4/C7/C9/C10 → inkonsistente Labels. Single-Source-of-Truth festlegen.
2. **Rate-Limit nicht verteilt** (`rate-limit.ts:10`): auf Multi-Instance umgehbar (`N × Limit`). Echter Schutz ist nur der Cosmos-Cost-Cap. Für harte Drosselung → verteilter Store oder uid-basiert.
3. **Token-Schätzung grob** (`cost-cap.ts:135-140`, `chars/4 + 3000`): kann reale Gemini-Kosten unterschätzen; kein Retro-Accounting. Reale Nutzung loggen + Limit kalibrieren.
4. **README veraltet** (Firebase Studio / Firebase Hosting / „Push to main → deploy“) — irreführend.
5. **LinkedIn produktiv unfertig:** Credentials + Referenzbilder Platzhalter; kein Token-Refresh.

Mittel:

6. **DSGVO-Rest (M7):** Gemini + Pinecone außerhalb EU. Mitigation heute: Client-Anonymisierung (default-on). Azure OpenAI `gpt-41-mini-prod` ist bereits provisioniert.
7. `mode=enforce` ≈ `warn` (Quality-Checks erzeugen kein `error`) — bewusster Expansion-Point, nicht offensichtlich kommuniziert.
8. **Read-Modify-Upsert ohne Concurrency-Control** (Rating/Profil/Token): „last write wins“.
9. **Cursor-Pagination auf `createdAt` ohne `id` -Tiebreaker** → Duplikat/Skip bei identischem Zeitstempel.
10. **Kein CI/CD + kein Deploy-Skript;** Quality-Harness nicht im Pre-Push-Gate.
11. **`competency_ratings` -Fallback** füllt 10 Defaults mit `score=null` statt zu dropfen — UI muss `competency_error` sichtbar machen, sonst wirken null-Scores wie „echte“ Nicht-Beobachtbarkeit.
12. **`pinecone-smoke`** liefert rohe Index-Treffer an jeden eingeloggten User (Index-Inhalt-Leak, Diagnose-Endpunkt) — in Produktion ggf. gaten.

Niedrig / Polish: Datumsformat im Dashboard hart `de-DE`; ICS-Erinnerung Timezone (immer 9:00 UTC); `evidenceNeedle` -Regex heuristisch; Floskel-Liste statisch (Code-Change zum Erweitern); LanguageSwitcher-Icon-Inkonsistenz (lucide vs. Material Icons); `next-auth` ist `beta.31` (auf finale Version heben, sobald verfügbar); `firebase-admin@13` transitiv (auf Security-Updates achten).

Bewusst & vertretbar (kein Bug): Prompt-Guard loggt statt blockt (Fencing ist die Maßnahme); offene Selbst-Provisionierung; `firebase-admin` transitiv via Genkit; Cost-Cap fail-open.

21. Prüf-Leitfaden für den Reviewer

Priorisierte Checkliste — was konkret zu verifizieren ist.

A. Sicherheit (zuerst):

- ☐ **Ownership lückenlos?** Jede daten-berührende Route ruft `checkSessionOwnership` vor dem Zugriff. Mit zweitem Test-Konto: fremde `sessionId / runId` → **403**, nicht 200.
- ☐ **IDOR wirklich zu?** `GET /runs/{sid}/{runId}` ohne Session liefert kein HTML mit `transcriptText / analysisJson`.
- ☐ **Auth vor Validierung?** Garbage-POST an `/api/analyze` ohne Session → **401** (nicht 400).
- ☐ **Secrets:** `git grep` nach Keys; Key-Vault-Referenzen „Resolved“; keine Secrets in `.env.local` im Repo.
- ☐ **LinkedIn-State:** Cookie-Vergleich **und** HMAC-Signatur; Token AES-256-GCM; Key-Rotation → sauberer Reconnect, kein Crash.
- ☐ **CSRF:** reichen `SameSite=lax` + NextAuth für die Custom-POST-Routen, oder ist ein expliziter Token nötig?
- ☐ `pinecone-smoke` in Produktion gewünscht/gegated?

B. Korrektheit der KI-Kette:

- ☐ **Schema-Forced Reasoning wirkt?** `npm run quality:regression -- --n=3` → Scores evidenzgestützt, stabil?
- ☐ **Grounding:** treten `EVIDENCE_NOT_GROUNDED` -Notes praktisch auf? (Logs.)
- ☐ **RAG-Truncation (4000 Zeichen)** bei langen Transkripten — Qualitätsverlust? Benchmarks.
- ☐ **Modell-Divergenz C1–C10** klären (autoritative Quelle).
- ☐ `lang -Enum de|en` vs. generisches `{{lang}}` im Prompt — Verhalten bei anderen Werten?
- ☐ `evidence.max(3)` vs. Prompt „1–2 Zitate“ — liefert das Modell oft 3?

C. Kosten & Robustheit:

- ☐ **Cost-Cap real?** `usage` -Container füllt sich pro User; `COST_CAP_MODE=enforce` + Limit überschritten → **429**; `ADMIN_EMAILS` -Bypass testen.
- ☐ **Rate-Limit-Realität** bei geplanter Instanzzahl bewerten (Single vs. Multi-Pod).
- ☐ **Cost-Cap fail-open** gegen Adversarial-Cosmos-Fehlerpfad prüfen.
- ☐ `Promise.allSettled` -**Degradation:** Basis-Fehler → 500, Kompetenz-Fehler → `competency_error` sichtbar im UI?

D. Daten & Persistenz:

- ☐ **ISO-Strings** statt Epoch in `createdAt` durchgängig.
- ☐ **Cursor-Pagination** mit identischem `createdAt` testen (Tiebreaker nötig?).

- ☐ **Partition-Key** `runs./sessionId` — alle Upserts nutzen `sessionId` (keine Cross-Partition-Queries).
- ☐ **TTL** am `usage` -Container aktiv (Datenwachstum begrenzt)?
- ☐ **Read-Modify-Upsert** (Rating/Profil): „last write wins“ akzeptabel/dokumentiert?

E. Betrieb & Qualität:

- ☐ `npm run typecheck` + `npm run build` grün (kein `ignoreBuildErrors`).
- ☐ `npm run quality:regression` → 5/5; mit `--judge` (Anthropic-Key) die 6 Dimensionen prüfen.
- ☐ Golden-Set-Coverage (nur 5 Szenarien; lange/gemischtsprachige Transkripte fehlen).
- ☐ README auf Azure aktualisieren; Deploy-Runbook in ein Skript gießen; Quality-Harness in CI.
- ☐ Azure-Monitor-Alerting auf `competency_error` / `quality_notes` -Counts?
- ☐ `next-auth` -Beta-Version + `firebase-admin` -Transitiv-Dep auf Security-Updates prüfen.

22. Anhang A — ENV-Variablen-Referenz

Alle in `.env.example` dokumentiert; geheime Werte in Produktion via Key Vault.

Variable	Zweck	Pflicht
<code>AUTH_URL</code> , <code>AUTH_TRUST_HOST</code> , <code>AUTH_SECRET</code>	NextAuth (exakte Origin, Proxy-Trust, Signaturschlüssel)	✓
<code>AUTH_MICROSOFT_ENTRA_ID_ID</code> / <code>_SECRET</code> / <code>_TENANT_ID</code>	Entra-Provider	✓
<code>COSMOS_ENDPOINT</code> , <code>COSMOS_KEY</code> , <code>COSMOS_DATABASE</code> (default <code>coach</code>)	Cosmos DB	✓
<code>GEMINI_API_KEY</code> , <code>GEMINI_TEXT_MODEL</code>	LLM	✓
<code>PINECONE_API_KEY</code> , <code>PINECONE_INDEX_HOST</code> , <code>PINECONE_NAMESPACE</code>	RAG	✓
<code>DAILY_TOKEN_LIMIT</code> (500000), <code>COST_CAP_MODE</code> (enforce), <code>ADMIN_EMAILS</code> (CSV)	Cost-Cap	–
<code>QUALITY_CHECKS_MODE</code> (warn)	Laufzeit-Qualitäts-Checks	–
<code>LINKEDIN_CLIENT_ID</code> / <code>_SECRET</code> , <code>LINKEDIN_REDIRECT_URI</code>	LinkedIn-OAuth	–
<code>LINKEDIN_TOKEN_KEY</code> , <code>LINKEDIN_BRAND_NAME</code> , <code>LINKEDIN_IMAGE_PERSON</code>	LinkedIn-Optionen	–
<code>NANOBANANA_MODEL</code>	Bild-Generierung	–
<code>ANTHROPIC_API_KEY</code> , <code>JUDGE_MODEL</code>	Quality-Loop LLM-Judge (nur Tooling)	–

Pflicht-Hinweis: `AUTH_SECRET` darf nicht leer sein; `AUTH_URL` muss exakt der Origin entsprechen (sonst NextAuth „UntrustedHost“/redirect-Mismatch); `AUTH_TRUST_HOST=true` ist hinter dem App-Service-Proxy zwingend.

23. Anhang B — Zentrale Prompts & Schemas (verbatim)

B.1 — Tailored-Feedback-Prompt (src/ai/flows/generate-tailored-feedback.ts:52-79 , vollständig):

```
You are an AI-powered communication coach for leadership conversations.

IMPORTANT RULES:
- Focus your evaluation primarily on the LEADER (manager).
- The transcript uses speaker labels. If leaderLabel/employeeLabel are provided, use them to i
- Do NOT reveal any internal sources, cards, vector DB, Pinecone, or metadata. Use relevant sn
- Do NOT use real names in quotes. Use the labels (leaderLabel / employeeLabel) or generic "Fü
- Output language: if lang is provided (e.g., "de"), write the feedback in that language. Othe

Transcript:
{{{inputText}}}

Conversation Type: {{{conversationType}}}
{{#if conversationSubType}}Conversation Subtype: {{{conversationSubType}}}{}/if}}
Goal: {{{goal}}}
{{#if lang}}Language: {{{lang}}}{}/if}}
{{#if jurisdiction}}Jurisdiction: {{{jurisdiction}}}{}/if}}
{{#if leaderLabel}}Leader Label: {{{leaderLabel}}}{}/if}}
{{#if employeeLabel}}Employee Label: {{{employeeLabel}}}{}/if}}

{{#if relevantSnippets}}
Internal Coaching Guidance (do not mention explicitly):
{{#each relevantSnippets}}
- {{{this}}}
{{/each}}
{{/if}}

Return ONLY the JSON fields required by the output schema.
```

B.2 — Tailored-Feedback-Schemas (:9-40):

```
GenerateTailoredFeedbackInput = {
  inputText, conversationType, conversationSubType?, goal,
  lang?, jurisdiction?, leaderLabel?, employeeLabel?, relevantSnippets?: string[]
}
GenerateTailoredFeedbackOutput = {
  summary: string, strengths: string[], improvements: string[],
  rewrites: string[], riskFlags: string[],
  scores: { overall?: 0..10 } & Record<string, number>    // .catchall(z.number())
}
```

B.3 — Kompetenz-Scoring: Schema (src/ai/flows/score-competencies.ts:17-48 , Feldordnung load-bearing):

```
const CompetencyRatingSchema = z.object({
  id: z.string(),
  name: z.string(),
  evidence: z.array(z.string()).max(3)
    .describe("1-2 wörtliche, anonymisierte Zitate AUS DEM TRANSKRIPT (max ~18 Wörter). " +
      "Keine Paraphrasen, keine erfundenen Zitate. Leeres Array, wenn nicht beobachtbar"),
  why: z.string()
    .describe("Begründung der Bewertung, ausschließlich auf die Evidenz gestützt. " +
      "'nicht ausreichend beobachtbar', wenn keine Evidenz vorliegt."),
  score: z.number().min(1).max(4).nullable()
    .describe("Ganzzahl 1-4 NUR wenn durch die Evidenz belegt; sonst null. 1=schwach ... 4=vorbildlich"),
  confidence: z.number().min(0).max(1).nullable().optional(),
});
```

B.4 — Kompetenz-Scoring: Prompt-Regeln (:54-87 , sinngemäß verbatim):

SPRACHE: Verfasse alle "why"-Begründungen in der Sprache des Transkripts (Zielsprache: {{lang}}
Bei lang="en" auf Englisch, bei "de" auf Deutsch. Evidenz-Zitate bleiben im Originalwortlaut.

SKALA 1-4: 1=schwach/kontraproduktiv ... 4=sehr gut/vorbildlich.

REIHENFOLGE (zwingend, pro Kompetenz):

- 1) Sammle zuerst die EVIDENCE: 1-2 wörtliche Zitate aus dem Transkript (anonymisiert).
- 2) Begründe (why) ausschließlich auf Basis dieser Zitate.
- 3) Vergib ERST DANN den score – nur wenn die Evidenz ihn belegt.

WENN NICHT ERKENNBAR: evidence=[], score=null, why="nicht ausreichend beobachtbar".
Erfinde KEINE Zitate. Ein Zitat MUSS WÖRTLICH und ZUSAMMENHÄNGEND im Transkript stehen.

KEIN ÜBER-SCORING: Reine Terminabsprachen, Logistik oder Small-Talk sind KEINE Führungsleistung.
Wenn eine Kompetenz nicht durch konkretes Führungsverhalten belegt ist, ist score = null.

B.5 — LLM-Judge-Dimensionen (scripts/quality/judge.ts:14-21): faithfulness , coverage , actionability , competency_consistency , tone , locale — je 1-5, n=3-Median, neutraler Claude-Judge.

24. Anhang C — Deploy-Runbook

```
# 1) Build (Standalone-Output)
cd C:\dev\AI_Coach
npm run build                # erzeugt .next/standalone (output:'standalone')

# 2) Assets ins Standalone-Verzeichnis (Pflicht – sonst fehlen static/public)
robocopy .next\static .next\standalone\next\static /E
robocopy public .next\standalone\public /E

# 3) ZIP packen (Inhalt von .next/standalone an die Zip-Wurzel) → coach-deploy.zip

# 4) Deploy auf Azure App Service
az webapp deploy -g pulsecraft-prod-rg -n pulsecraft-coach \
  --src-path coach-deploy.zip --type zip --async false

# 5) Verifikation
curl -s -o NUL -w "%{http_code}" https://pulsecraft-coach.azurewebsites.net/analyze # 200
# geschützte API ohne Session → 401 ; az webapp log tail → sauber

# Qualitäts-Gate (lokal, vor Deploy empfohlen)
npm run quality:regression # 5/5 PASS erwartet ; -- --n=3 für Stabilität
```

App-Settings (Auszug, via Key-Vault-Referenz): AUTH_*, COSMOS_*, GEMINI_*, PINECONE_*, COST_CAP_MODE=enforce, DAILY_TOKEN_LIMIT=500000, ADMIN_EMAILS, QUALITY_CHECKS_MODE=warn, WEBSITE_RUN_FROM_PACKAGE=1, ENABLE_ORYX_BUILD=false, PORT=8080, Startup node server.js.

25. Anhang D — Glossar & Datei-Landkarte

Glossar: *Run* = ein Analyse-Lauf (1 Transkript → 1 Report); *Session* = Container für mehrere Runs eines Nutzers; *uid* = Entra-OIDC- sub , App-weites Identitäts-/Ownership-Kriterium; *Kompetenz (C1–C10)* = Leadership-Dimension, Skala 1–4 oder null; *QualityNote* = deterministischer Qualitäts-Befund; *Golden-Set* = synthetische Test-Transkripte mit erwarteten Eigenschaften; *Cost-Cap* = Tages-Token-Budget pro User in Cosmos.

Datei-Landkarte (Einstiegspunkte für den Code-Review):

Bereich	Schlüssel-Dateien
Auth	<code>src/auth.ts</code> , <code>src/lib/api-auth.ts</code> , <code>src/lib/api-client.ts</code> , <code>src/providers/auth-provider.tsx</code> , <code>src/middleware.ts</code>
Daten	<code>src/lib/cosmos.ts</code> , <code>src/lib/server/runs-store.ts</code>
API	<code>src/app/api/analyze/route.ts</code> , <code>src/app/api/runs/*/route.ts</code> , <code>src/app/api/users/profile/route.ts</code>
KI	<code>src/ai/genkit.ts</code> , <code>src/ai/flows/{generate-dynamic-feedback,generate-tailored-feedback,score-competencies}.ts</code> , <code>src/lib/pinecone.ts</code> , <code>src/lib/prompt-guard.ts</code>
Kosten/Qualität	<code>src/lib/server/cost-cap.ts</code> , <code>src/lib/rate-limit.ts</code> , <code>src/lib/quality-core.ts</code> , <code>src/lib/server/quality-checks.ts</code> , <code>scripts/quality/*</code>
Frontend	<code>src/app/analyze/AnalyzeClient.tsx</code> , <code>src/components/app/report-dashboard.tsx</code> , <code>src/lib/transcript-utils.ts</code> , <code>src/lib/report-utils.ts</code> , <code>src/lib/pdf/parsePdfToText.ts</code>
LinkedIn	<code>src/app/api/linkedin/*/route.ts</code> , <code>src/lib/linkedin.ts</code> , <code>src/lib/server/linkedin-connection.ts</code> , <code>src/lib/nanobanana.ts</code>
Infra/Doku	<code>next.config.ts</code> , <code>tsconfig.json</code> , <code>.env.example</code> , <code>docs/azure-migration/PLAN.md</code> , <code>docs/quality/README.md</code>

Erstellt 2026-06-13 aus dem laufenden Code (Branch `main`, nach Merge von Quality-Loop Q2). Alle `Datei:Zeile` -Verweise beziehen sich auf diesen Stand. Schnellster Einstieg in eine kritische Bewertung: Abschnitt 21 (Prüf-Leitfaden) + Abschnitt 20 (Schwächen), dann die Datei-Landkarte oben.